

CrediPredict: ML-Powered Credit Card Approval Predictor

1. Executive Summary

In the modern financial sector, traditional credit assessment methods are often slow, prone to human bias, and inefficient at handling high volumes of applications. **CrediPredict** addresses this challenge by introducing an automated, data-driven machine learning system designed to evaluate credit card applications instantly. By analyzing key risk indicators such as income, outstanding debt, employment history, and prior defaults, the system classifies applicants into "Approved" or "Rejected" risk profiles. The core architecture relies on a highly optimized Random Forest Classifier wrapped in a lightweight Flask web framework and made publicly accessible via Ngrok.

2. Problem Statement & Scope

Lending institutions must balance maximizing card issuance revenue with minimizing the risk of financial default. Evaluating these metrics manually creates operational bottlenecks.

2.1 Project Scope

- **Data Analysis:** Identify significant features that correlate heavily with credit risk.
- **Predictive Modeling:** Train and serialize a binary classification model reaching > 85% accuracy and high recall for default detection.
- **Web Application Interface:** Build a responsive, user-friendly frontend to capture applicant data dynamically.
- **Deployment:** Create a secure tunnel to expose the local runtime environment to external evaluators.

3. System Requirements & Tech Stack

3.1 Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux (Ubuntu 20.04+)
- **Language Runtime:** Python 3.9+
- **Core Libraries:**
 - Flask==3.0.x (Backend Web Framework)
 - scikit-learn==1.4.x (Machine Learning Engine)

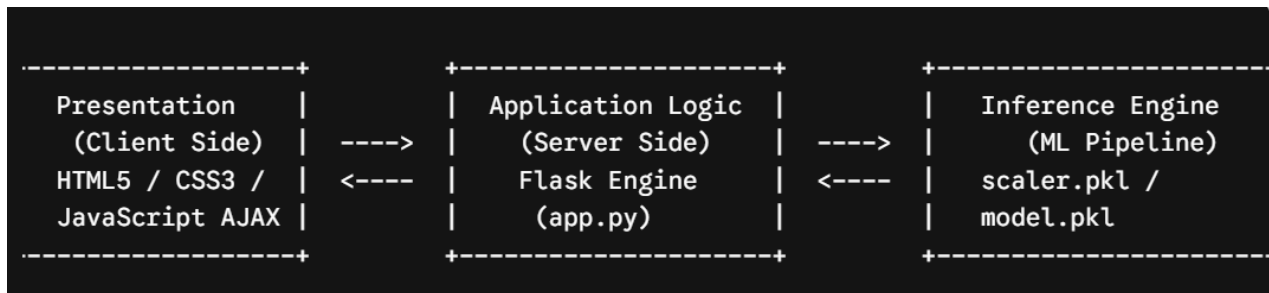
- pandas==2.2.x & numpy==1.26.x (Data Manipulation)
- joblib==1.3.x (Model Serialization)

3.2 Hardware Requirements (Minimum Recommended)

- **Processor:** Intel Core i5 or AMD Ryzen 5 (Quad-Core minimum)
- **Memory (RAM):** 8 GB RAM
- **Storage:** 500 MB of free storage space for dependencies and datasets

4. Technical Architecture

The system employs a standard three-tier decoupled architecture to ensure independent data processing, application logic execution, and user presentation.



4.1 Execution Flow

1. **Input:** The user inputs details into the frontend UI form.
2. **Request:** JavaScript intercepts the submission and sends an asynchronous POST JSON request to the /predict route.
3. **Validation & Processing:** The Flask backend verifies the data types and passes the numerical array into the ML inference pipeline.
4. **Transformation & Inference:** Features are normalized using a pre-saved StandardScaler. The serialized classification model calculates the final decision.
5. **Response:** The frontend receives the response payload containing the approval status, confidence levels, and risk parameters, updating the UI dynamically without refreshing the page.

5. Machine Learning Methodology

5.1 Mathematical Foundation

The prediction system computes the conditional probability of approval given an applicant's profile vector X . The final decision boundary relies on a calibrated threshold θ :

$$\text{Decision}(X) = \begin{cases} \text{Approved,} & \text{if } P(Y = 1 | X) \geq \theta \\ \text{Rejected,} & \text{if } P(Y = 1 | X) < \theta \end{cases}$$

Where:

- $X = [\text{Age}, \text{Income}, \text{Debt}, \text{Employment Duration}, \text{Prior Defaults}]$
- $\theta = 0.50$ (Standard classification cutoff)

To evaluate the mathematical rigor of the model during validation, the system measures Precision, Recall, and the harmonic mean (F_1 -score):

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$F_1\text{-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

5.2 Feature Mapping Matrix

Feature Name	Data Type	Description	Target Impact
Income	Float / Continuous	Annual earnings of the applicant	High positive correlation
Age	Float / Continuous	Age in years	Low correlation

Feature Name	Data Type	Description	Target Impact
Debt	Float / Continuous	Total outstanding liability ratio	High negative correlation
Years Employed	Float / Continuous	Duration with current employer	Moderate positive correlation
Prior Default	Binary (0 / 1)	Past payment failure record	Extreme negative impact

6. Implementation Code

6.1 Backend Controller (app.py)

```
import joblib

import numpy as np

from flask import Flask, request, jsonify, render_template

app = Flask(__name__)

# Core Artifact Loading
try:
    model = joblib.load("model.pkl")
    scaler = joblib.load("scaler.pkl")
except Exception as e:
    print(f"Critical Error: Failed to load serialized binaries. {str(e)}")

@app.route("/")
def index():
```

```
return render_template("index.html")

@app.route("/predict", methods=["POST"])
def predict():
    try:
        data = request.get_json()

        # Extract features
        income = float(data.get("income", 0))
        age = float(data.get("age", 0))
        debt = float(data.get("debt", 0))
        years_employed = float(data.get("years_employed", 0))
        prior_default = int(data.get("prior_default", 0))

        # Validation checks
        if income < 0 or age < 18 or debt < 0:
            return jsonify({"success": False, "error": "Invalid constraint inputs."}), 400

        # Structure payload (Must mirror training layout exactly)
        raw_features = np.array([[age, income, years_employed, prior_default, debt]])
        scaled_features = scaler.transform(raw_features)

        # Evaluate model prediction
        prediction = int(model.predict(scaled_features)[0])
        probabilities = model.predict_proba(scaled_features)[0]
        confidence = float(probabilities[prediction]) * 100
```

```
return jsonify({  
    "success": True,  
    "prediction": "Approved" if prediction == 1 else "Rejected",  
    "confidence": round(confidence, 2),  
    "risk_score": round(float(probabilities[0]) * 100, 2)  
})  
  
except Exception as e:  
    return jsonify({"success": False, "error": str(e)}), 500  
  
if __name__ == "__main__":  
    app.run(port=5050, debug=True)
```

7. Deployment & Environment Setup

7.1 Local Deployment Execution

1. Initialize a virtual environment to shield packages from conflicts:
2. Install all required project dependencies:
3. Launch the local Flask application listener:

7.2 Secure Public Tunneling via Ngrok

To present the product to external remote stakeholders without deploying to a costly cloud provider:

1. Install Ngrok globally or via pip:
2. Link the production workspace to your verified account token:
3. Forward traffic from port 5050 out to the public web server:
4. Copy the generated secure <https://> routing link to access the platform from any internet-connected device.

8. Conclusion & Future Enhancements

The **CrediPredict** application successfully transitions a predictive machine learning asset out of isolated Jupyter Notebook scripts into a tangible, high-velocity business automation tool.

Future System Roadmap

- **Database Auditing Layer:** Integrate an object-relational mapper (SQLAlchemy) alongside an SQLite or PostgreSQL database backend to record prediction requests for subsequent model drift evaluations.
- **Explainable AI (XAI):** Embed a local framework hook utilizing SHAP (SHapley Additive exPlanations) to dynamically map and explain *why* an applicant was rejected right inside the front-end dashboard UI.
- **Alternative Data Pipelines:** Incorporate utility bills and rent payment metrics to enhance credit scoring accuracy for younger applicants lacking an extensive credit history.

